Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

Разработка учебной системы программирования

Компилятор с ЯВУ

Дисциплина: «Системы программирования»

Вариант № 11

Исполнители:
Студенты группы 5140904/50102 Сырбу Г. В.
Федерова С. К.

Руководитель: Расторгуев В. Я.

Санкт-Петербург
2026

Оглавление

1. Задание на курсовую работу.....3
2. Постановка задачи модификации макета в части компилятора с ЯВУ.....5
3. Модификация состава и структуры БД макета компилятора с ЯВУ.....8
3.1. Модификация БД лексического анализатора.....8
3.2. Модификация БД синтаксического анализатора.....9
4. Модификация алгоритма макета компилятора с ЯВУ.....18
5. Результаты экспериментальной проверки модифицированного компилятора с ЯВУ.....19
6. Выводы по результатам работы.....19

1. Задание на курсовую работу

Задание на КР дано в следующей общей формулировке:

«Необходимо выполнить доработку элементов макета учебной системы программирования до уровня, позволяющего обрабатывать «новые» для макета конструкции языка высокого уровня, примененные в соответствующем варианте».

Общая формулировка персонализирована в варианте № 11, приведенном на рис. 1.

```
EX11: PROC OPTIONS ( MAIN );
DCL A BIN FIXED INIT ( 3 );
DCL B DEC FIXED INIT ( 3 );
DCL C BIN FIXED;
C = A = ( B * 5 );
END EX11;
```

Рис. 1. Персональный вариант № 11

По договорённости с преподавателем в персональный вариант были внесены изменения, касающиеся использования скобок в арифметических выражениях: вложенное выражение со скобками было разделено на два последовательных оператора присваивания.

В результате была сформулирована следующая входная программа (рис. 2):

```
EX11: PROC OPTIONS ( MAIN );
DCL A BIN FIXED INIT ( 3 );
DCL B DEC FIXED INIT ( 3 );
DCL C BIN FIXED;
C = B * 5;
C = A = C;
END EX11;
```

Рис. 2. Модифицированный персональный вариант № 11 (согласованный с преподавателем)

Схема функционирования макета учебной системы программирования, являющегося, согласно заданию на КР, одним из элементов исходных данных, приведена на рис. 3.

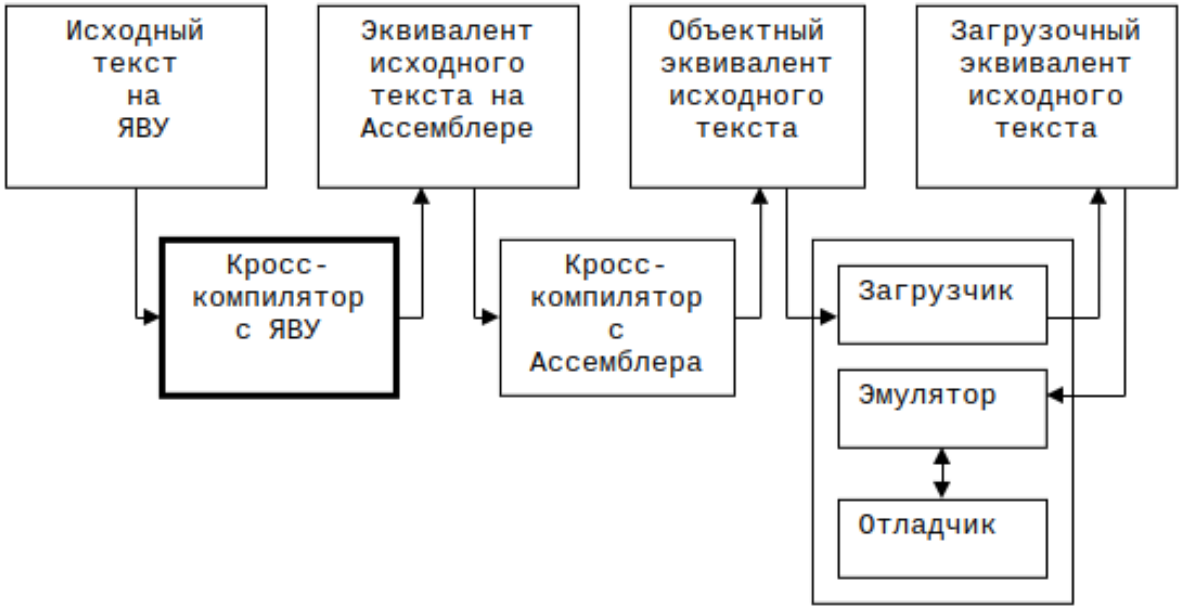

Рис. 3. Схема функционирования учебной системы программирования

Дополнительные требования к КР, вытекающие из рис.2, предполагают то, что данная система программирования работает на технологической ЭВМ (IBM PC) и является по существу кросс-системой для объектной ЭВМ (IBM 370). В этой системе:

- в качестве языка высокого уровня (ЯВУ) выбран язык, образованный из подмножества языковых конструкций ПЛ1, а исходная программа готовится в виде текстового файла технологической ЭВМ с расширением *.pli
- язык АССЕМБЛЕРА сформирован из языковых конструкций АССЕМБЛЕРА IBM 370, а ассемблеровский эквивалент исходной программы формируется в виде текстового файла технологической ЭВМ с расширением *.ass
- объектный эквивалент исходной программы готовится в формате объектных файлов операционной системы ОС IBM 370 и хранится в виде двоичного файла технологической ЭВМ с расширением *.tex

- загрузочный эквивалент исходной программы представляет собой машинный код IBM 370, запоминаемый в области ОЗУ технологической ЭВМ, являющейся зоной загрузки для эмулятора объектной ЭВМ

На рис. 2 жирным контуром выделен компилятор с ЯВУ, являющийся тем элементом макета, результаты модификации которого под персональный вариант No XX (см. рис 1) приведены в данном отчете.

2. Постановка задачи модификации макета в части компилятора с ЯВУ

Схема функционирования модифицированного макетного компилятора с ЯВУ должна соответствовать рис. 4.

Рис. 4. Схема функционирования модифицированного макетного компилятора с ЯВУ

Функционал модифицированного макетного компилятора должен обеспечить компиляцию исходного текста на ЯВУ (см. рис. 1) в эквивалент исходного текста на Ассемблере (см. рис. 5)

Метка	КОП	Операнды	Пояснение
EX11	START	0	Program start. The relative address counter is initialized to 0.
	BALR	@RBASE,0	Load the current program address into the base register @RBASE.
	USING	*,@RBASE	Declare @RBASE as the base register for addressing program objects.
	ZAP	@BUF(8),B(3)	Copy the packed decimal value of B into the working buffer @BUF.

Метка	КОП	Операнды	Пояснение
	MP	@BUF(8),@FIVE(1)	Multiply the decimal value in @BUF by the constant 5.
	CVB	@R1,@BUF	Convert the packed decimal result in @BUF to binary and place it in register @R1.
	STH	@R1, C	Store the binary result from register @R1 into variable C.
	LH	@R2,A	Load the binary value of variable A into register @R2.
	CH	@R2,C	Compare the value in register @R2 with the value stored in C.
	BC	8, @TRUE	Branch to label @TRUE if the values are equal.
	LA	@R3,0	Load value 0 into register @R3 (boolean FALSE).
	B	@SAVE	Unconditional branch to label @SAVE.
@TRUE	LA	@R3,1	Load value 1 into register @R3 (boolean TRUE).
@SAVE	STH	@R3,C	Store the boolean result (0 or 1) into variable C.
	BCR	15,@RVIX	Return control to the calling program using the return register.
A	DC	H'3'	Declare binary variable A (halfword) initialized with value 3.
B	DC	PL3'3'	Declare decimal variable B in packed decimal format with value 3.

Метка	КОП	Операнды	Пояснение
C	DS	H	Reserve storage for binary variable C (halfword).
@FIVE	DC	PL1'5'	Declare packed decimal constant with value 5.
	DS	0F	Align the next data item to a fullword boundary.
@BUF	DC	PL8'0'	Working buffer used for packed decimal arithmetic operations.
@RBASE	EQU	5	Assign register number 5 as the base register.
@R1	EQU	6	Register used for decimal-to-binary conversion.
@R2	EQU	2	Register used for comparison operations.
@R3	EQU	3	Register used to store the boolean result.
@RVIX	EQU	14	Register used to return control to the operating system.
	END		End of the assembly program.

Рис. 5. Эталон ассемблерного эквивалента персонального варианта № 11

3. Модификация состава и структуры БД макета компилятора с ЯВУ

3.1. Модификация БД лексического анализатора

БД лексического анализатора определяется путем выделения подмножества терминальных символов, в которое входят все терминальные символы, являющиеся разделителями термов языка.

Для персонального варианта (см. рис. 1) разделителями термов являются терминальные символы из множества:

{":", "(", ")", ";", "+", "-", "=", "_", "*"}

где модификация лексических настроек демо-примера состоит в расширении списка символов-разделителей термов дополнительными символами звездочка “*”

Т.к. авторы макета лексического анализатора разместили подмножество символов-разделителей термов прямо в программе лексического анализатора compress_ISXTXT(), а не в специальной таблице, то коррекция БД лексического анализатора выполнена в теле этой программы:

```
void compress_ISXTXT()
{
  if (
    ( ISXTXT[I1][I2] == '+' ||
      ISXTXT[I1][I2] == '−' ||
      ISXTXT[I1][I2] == '=' ||
      ISXTXT[I1][I2] == '(' ||
      ISXTXT[I1][I2] == ')' ||
      ISXTXT[I1][I2] == '*' /* <− добавлен разделитель '*' */
    )
    && PREDSYM == ' '
  )
  { I3−−; goto L1; }

  if ( ISXTXT[I1][I2] == ' ' &&
     ( PREDSYM == '+' || PREDSYM == '−' ||
       PREDSYM == '=' || PREDSYM == '*' /* <− добавлен '*' */
     )
  )
  { goto L2; }
}
```

Рис 6. Фрагмент compress_ISXTXT() с добавленным символом «*»

3.2. Модификация БД синтаксического анализатора

Высокоуровневое определение модифицированного синтаксиса для персонального варианта (см. рис. 1) с выделением модификаций жирным шрифтом в определение нетерминала <ZNK> приведен ниже:

1. ⟨PRO⟩ ::= ⟨OPR⟩⟨TEL⟩⟨OEN⟩

2. ⟨OPR⟩ ::= ⟨IPR⟩:PROC_OPTIONS(MAIN);

3. ⟨IPR⟩ ::= ⟨IDE⟩

4. ⟨IDE⟩ ::= ⟨BUK⟩ | ⟨IDE⟩⟨BUK⟩ | ⟨IDE⟩⟨CIF⟩

5. ⟨BUK⟩ ::= A | B | C | D | E | M | P | X

6. ⟨CIF⟩ ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

7. ⟨TEL⟩ ::= ⟨ODC⟩ | ⟨TEL⟩⟨ODC⟩ | ⟨TEL⟩⟨OPA⟩ | **⟨TEL⟩⟨OPL⟩**

8. ⟨ODC⟩ ::= DCL_⟨IPE⟩_BIN_FIXED(⟨RZR⟩);

9. | DCL_⟨IPE⟩_BIN_FIXED(⟨RZR⟩)INIT(⟨LIT⟩);

   | DCL_⟨IPE⟩_BIN_FIXED;

   | DCL_⟨IPE⟩_BIN_FIXED_INIT(⟨LID⟩);

   | DCL_⟨IPE⟩_DEC_FIXED_INIT(⟨LID⟩);

10. ⟨IPE⟩ ::= ⟨IDE⟩

11. ⟨RZR⟩ ::= ⟨CIF⟩ | ⟨RZR⟩⟨CIF⟩

12. ⟨LIT⟩ ::= ⟨MAN⟩B

13. ⟨MAN⟩ ::= 1 | ⟨MAN⟩0 | ⟨MAN⟩1

14. **⟨LID⟩ ::= ⟨CIF⟩ | ⟨LID⟩⟨CIF⟩**

15. ⟨OPA⟩ ::= ⟨IPE⟩=⟨AVI⟩;

16. ⟨AVI⟩ ::= ⟨LIT⟩ | **LID** | ⟨IPE⟩| ⟨AVI⟩⟨ZNK⟩⟨LIT⟩ | **AVI ZNK LID** | ⟨AVI⟩⟨ZNK⟩⟨IPE⟩

17. **⟨AVL⟩ ::= ⟨AVI⟩⟨ZNL⟩⟨LIT⟩ | ⟨AVI⟩⟨ZNL⟩⟨LID⟩ | ⟨AVI⟩⟨ZNL⟩⟨IPE⟩**

18. **⟨OPL⟩ ::= ⟨IPE⟩=⟨AVL⟩;**

19. ⟨ZNK⟩ ::= + | - | *****

20. **⟨ZNL⟩ ::= =**

21. ⟨OEN⟩ ::= END_⟨IPR⟩

Здесь использованы следующие метасимволы и символы:

"<" и ">" - левый и правый ограничители нетерминального символа,

"::=" - метасимвол со смыслом "равно по определению",

"|" - метасимвол альтернативного определения "или",

"_" - терминальный символ со смыслом "пробел",

"<PRO>" - нетерминал "программа",

"<OPR>" - нетерминал "оператор пролога программы",

"<IPR>" - нетерминал "имя программы",

"<IDE>" - нетерминал "идентификатор",

"<BUK>" - нетерминал "буква",

"<CIF>" - нетерминал "цифра",

"<TEL>" - нетерминал "тело программы",

"<ODC>" - нетерминал "оператор declare",

"<IPE>" - нетерминал "имя переменной",

"<RZR>" - нетерминал "разрядность",

"<LIT>" - нетерминал "литерал",

"<MAN>" - нетерминал "мантисса",

"<LID>" — нетерминал "десятичный целочисленный литерал"

"<OPA>" - нетерминал "оператор присваивания арифметический",

"<OPL>" — нетерминал "оператор присваивания с логическим выражением"

"<AVI>" - нетерминал "арифметическое выражение",

"<AVL>" — нетерминал "логическое выражение"

"<ZNK>" - нетерминал "знак",

"<ZNL>" — нетерминал "знак логической операции"

"<OEN>" - нетерминал "оператор эпилога программы".

Сначала перепишем каждое правило грамматики, заменяя формат метаязыка Бэкуса на формат продукций (или на форму распознавания).

Для этого в каждой альтернативе каждого правила в формате метаязыка Бэкуса поменяем местами правую и левую части, изменив при этом символ "::=" на "──>". Получим следующий набор продукций:

1. <OPR><TEL><OEN> ──> <PRO>
2. <IPR>:PROC_OPTIONS(MAIN); ──> <OPR>
3. <IDE> ──> <IPR>
4. <BUK> ──> <IDE>

5. <IDE><BUK> ⟶ <IDE>

6. <IDE><CIF> ⟶ <IDE>

7. A ⟶ <BUK>

8. B ⟶ <BUK>

9. C ⟶ <BUK>

10. D ⟶ <BUK>

11. E ⟶ <BUK>

12. M ⟶ <BUK>

13. P ⟶ <BUK>

14. X ⟶ <BUK>

15. 0 ⟶ <CIF>

16. 1 ⟶ <CIF>

17. 2 ⟶ <CIF>

18. 3 ⟶ <CIF>

19. 4 ⟶ <CIF>

20. 5 ⟶ <CIF>

21. 6 ⟶ <CIF>

22. 7 ⟶ <CIF>

23. 8 ⟶ <CIF>

24. 9 ⟶ <CIF>

25. <ODC> ⟶ <TEL>

26. <TEL><ODC> ⟶ <TEL>

27. <TEL><OPA> ⟶ <TEL>

28. <TEL><OPL> ⟶ <TEL>

29. DCL_<IPE>_BIN_FIXED(<RZR>); ⟶ <ODC>

30. DCL_<IPE>_BIN_FIXED(<RZR>)INIT(<LIT>); ⟶ <ODC>

31. DCL_<IPE>_BIN_FIXED; ⟶ <ODC>

32. DCL_<IPE>_BIN_FIXED_INIT(<LID>); ⟶ <ODC>

33. DCL_<IPE>_DEC_FIXED_INIT(<LID>); ⟶ <ODC>
34. <IDE> ⟶ <IPE>
35. <CIF> ⟶ <RZR>
36. <RZR><CIF> ⟶ <RZR>
37. <MAN>B ⟶ <LIT>
38. 1 ⟶ <MAN>
39. <MAN>0 ⟶ <MAN>
40. <MAN>1 ⟶ <MAN>
41. <CIF> ⟶ <LID>
42. <LID><CIF> ⟶ <LID>
43. <IPE>=<AVI>; ⟶ <OPA>
44. <LIT> ⟶ <AVI>
45. <LID> ⟶ <AVI>
46. <IPE> ⟶ <AVI>
47. <AVI><ZNK><LIT> ⟶ <AVI>
48. <AVI><ZNK><LID> ⟶ <AVI>
49. <AVI><ZNK><IPE> ⟶ <AVI>
50. <AVI><ZNL><LIT> ⟶ <AVL>
51. <AVI><ZNL><LID> ⟶ <AVL>
52. <AVI><ZNL><IPE> ⟶ <AVL>
53. <IPE>=<AVL>; ⟶ <OPL>
54. + ⟶ <ZNK>
55. - ⟶ <ZNK>
56. * ⟶ <ZNK>
57. = ⟶ <ZNL>
58. END_<IPR> ⟶ <OEN>

Теперь, просматривая каждую из продукций слева-направо, сгруппируем продукции, имеющие общие части в "кусты", в которых роль "ствола" играют

общие части продукций, а роль "ветвей" – различающиеся части продукций. Получим следующий результат:

1. Куст идентификаторов и букв:

- A, B, C, D, E, M, P, X —> <BUK>
- <BUK> —> <IDE>
- <IDE>
    - <BUK> —> <IDE>
    - <CIF> —> <IDE>
- <IDE> —> <IPE>
- <IDE> —> <IPR>

2. Куст чисел и литералов:

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 —> <CIF>
- 1 —> <MAN>
- <MAN>
    - B —> <LIT>
    - 0 —> <MAN>
    - 1 —> <MAN>
- <CIF>
    - —> <RZR>
    - —> <LID>
- <RZR><CIF> —> <RZR>
- <LID><CIF> —> <LID>

3. Куст оператора объявления (DCL):

- DCL_ <IPE>_
    - BIN_FIXED
        - (<RZR>)
            - ; —> <ODC>

            - INIT(<LIT>); ⟶ <ODC>
        - ; ⟶ <ODC>
        - _INIT(<LID>); ⟶ <ODC>
    - DEC_FIXED_INIT(<LID>); ⟶ <ODC>

4. Куст выражений (арифметических и логических):
    - <LIT> ⟶ <AVI>
    - <LID> ⟶ <AVI>
    - <IPE> ⟶ <AVI>
    - <AVI>
        - <ZNK>
            - <LIT> ⟶ <AVI>
            - <LID> ⟶ <AVI>
            - <IPE> ⟶ <AVI>
        - <ZNL>
            - <LIT> ⟶ <AVL>
            - <LID> ⟶ <AVL>
            - <IPE> ⟶ <AVL>

5. Куст операторов присваивания:
    - <IPE>=
        - <AVI>; ⟶ <OPA>
        - <AVL>; ⟶ <OPL>

6. Куст знаков и разделителей:
    - +, -, * ⟶ <ZNK>
    - = ⟶ <ZNL>

7. Куст структуры программы (Тело):
    - <ODC> ⟶ <TEL>
    - <TEL>

- <ODC> ──> <TEL>
- <OPA> ──> <TEL>
- <OPL> ──> <TEL>

8. Куст корня программы:

- <IPR>:PROC_OPTIONS(MAIN); ──> <OPR>
- <OPR><TEL><OEN> ──> <PRO>
- END_<IPR> ──> <OEN>

1. Обработка объявлений BIN FIXED (Разбор базовой и упрощенной формы)

Ранее макет требовал обязательного указания разрядности. Для вашего варианта добавлены альтернативные переходы (ALT) на изолированную цепочку 232.

code C

```
/* Записи 59–73: Основная ветвь DCL ... BIN FIXED */
59  { 60,  58,  "( ",  232 }, // Если нет "(", уходим на обработку без
разрядности
60  { 61,  59,  "RZR", 0   },
61  { 62,  60,  ") ",  0   },
62  { 63,  61,  "; ",  65  }, // Если нет ";", пробуем ветку INIT (запись 65)
63  { 64,  62,  "ODC", 0   },
64  { 0,   63,  "* ",  0   },
65  { 66,  61,  "I ",  0   }, // Разбор слова INIT
...
73  { 64,  72,  "ODC", 0   },

/* Записи 232–246: Изолированная цепочка для BIN FIXED без разрядности */
232 { 233, 58,  " ",   233 }, // Обработка пробела после FIXED
233 { 234, 58,  "I ",  241 }, // Если видим "I" (INIT), идем на 234, иначе на
241 (конец)
234 { 235, 233, "N ",  0   },
235 { 236, 234, "I ",  0   },
236 { 237, 235, "T ",  0   },
237 { 238, 236, "( ",  0   },
238 { 239, 237, "LIT", 240 }, // Модификация: на ALT 240 добавлен разбор
десятичного RZR
239 { 242, 238, ") ",  0   },
240 { 239, 237, "RZR", 0   },
241 { 63,  58,  "; ",  0   }, // Возврат в штатную точку завершения ODC
```

2. Обработка объявлений DEC FIXED (Ваш специфический тип)

Запись 187 перехватывает управление, если вместо ожидаемого CHAR (демо-вариант) встречается буква D (DEC).

code C
```
/* Записи 187, 201–210: Ветка DEC FIXED */
187 { 188, 49,  "C ",  201 }, // Если не "C" (CHAR), пробуем 201 (DEC)
201 { 202, 49,  "D ",  0   },
202 { 203, 201, "E ",  0   },
203 { 204, 202, "C ",  0   },
204 { 205, 203, " ",   0   },
205 { 206, 204, "F ",  0   },
206 { 207, 205, "I ",  0   },
207 { 208, 206, "X ",  0   },
208 { 209, 207, "E ",  0   },
209 { 210, 208, "D ",  0   },
210 { 211, 209, "( ",  250 }, // Если нет "(", уходим на INIT (запись 250)

/* Записи 250–265: Цепочка для DEC FIXED INIT(...) */
250 { 251, 209, "",    251 },
251 { 252, 209, "I ",  257 },
256 { 259, 255, "LIT", 258 }, // Ожидание литерала
257 { 214, 209, "; ",  0   }, // Завершение оператора
```

3. Арифметические и логические выражения (Умножение и знак "=")

Здесь реализовано ваше ограничение: символ * ведет к знаку операции, а второй знак = в выражении распознается как операция сравнения.

code C
```
/* Записи для разбора AVI и логики */
168 { 169, 165, "IPE", 229 }, // Правый операнд: имя или число (через 229)
198 { 199, 0,   "* ",  0   }, // Терминал умножения для ZNK
199 { 200, 198, "ZNK", 0   },
226 { 227, 0,   "= ",  0   }, // Терминал равенства для ZNL (сравнение)
227 { 228, 226, "ZNK", 0   },
229 { 230, 165, "RZR", 0   }, // Обработка десятичного операнда (как RZR)
230 { 231, 229, "AVI", 0   }, // Свертка в арифметическое выражение
```

4. Замыкание тела программы (Связь с логом)

Запись 186 изменена, чтобы после каждого оператора (ODC, OPA, OPL) автомат возвращался к разбору тела программы (TEL).

code C
```
186 { 0, 181, "* ", 0 }, // Возврат к корректному предшественнику TEL
```

4. Модификация алгоритма макета компилятора с ЯВУ

Модификация алгоритма макета компилятора с ЯВУ была произведена параметрически путем модификации той части параметров настройки компилятора ЯВУ, которая относится к семантике ЯВУ. Этой частью семантических параметров является библиотека семантических подпрограмм. Объем модификаций этой библиотеки в общем случае определяется объемом модификации синтаксиса ЯВУ. Для условного персонального варианта №11 модификация синтаксиса (см. раздел 3) привела к необходимости модификации алгоритма макета компилятора с ЯВУ в части семантической подпрограммы komppl.c

```
  {/*.  54      .*/   "LID" , 201 , 'N' }, /* Десятичный литерал */
  {/*.  55      .*/   "AVL" , 205 , 'N' }, /* Логическое выражение */
  {/*.  56      .*/   "OPL" , 210 , 'N' }, /* Оператор присваивания (логич) */
  {/*.  57      .*/   "ZNL" , 215 , 'N' }  /* Знак логической операции (=) */

int LID1() { return 0; }
int LID2() { return 0; }

int ZNL1() { return 0; }
int ZNL2() { return 0; }

int AVL1() { return 0; }
int AVL2() {
    char i;
    FORM(); /* Разбиваем на лексемы: [AVI] [ZNL] [LIT/LID/IPE] */

    memcpy(ASS_CARD._BUFCARD.OPERAC, "C", 1);
    strcpy(ASS_CARD._BUFCARD.OPERAND, "RRAB,");
    strcat(ASS_CARD._BUFCARD.OPERAND, FORMT[2]);

    SETCOMM("Logical comparison");
    ZKARD();
    return 0;
}

int OPL1() { return 0; }
int OPL2() {
    SETCOMM("Store logical result");
    return 0;
}

int (* FUN [NNETRM][2]) () =
   {
     ...
     { LID1, LID2 },
```

```
    { AVL1, AVL2 },
    { OPL1, OPL2 },
    { ZNL1, ZNL2 }
  };
```

5. Результаты экспериментальной проверки модифицированного компилятора с ЯВУ

Работоспособность модифицированного под персональный вариант макета компилятора с ЯВУ была подтверждена испытательным прогоном его в среде ОС Linux. В результате на выходе был получен текст на Ассемблере, который совпал с текстом, приведенным на рис. 4.

6. Выводы по результатам работы

Порученное задание выполнено успешно. Успешность выполнения порученного задания подтверждена полным совпадением выходного текста, полученного при экспериментальной проверке компилятора с ЯВУ, с эталоном на рис. 4.